

RHCSA EX200 — Resumo Dia 05/05

SELinux Completo · Firewall · Apache Porta Não-padrão

PROVA: 14/05

4 Pilares do SELinux no EX200

Pilar	Comando-chave
Modo global	<code>setenforce + /etc/selinux/config</code>
Contexto de arquivo	<code>semanage fcontext + restorecon</code>
Boolean	<code>setsebool -P BOOL 1</code>
Porta não-padrão	<code>semanage port -a -t TIPO -p tcp PORTA</code>

Regra dos Dois Passos — vale para SELinux E Firewall

Camada	Ativo agora	Persistente (reboot)
SELinux	<code>setenforce 1</code>	<code>SELINUX=enforcing</code> no config
Firewall	<code>firewall-cmd (sem --permanent)</code>	<code>--permanent + --reload</code>

BLOCO 1 — SELinux: Modo Enforcing / Permissive

A Regra dos Dois Passos: SELinux tem duas camadas independentes que precisam ser configuradas juntas. `setenforce` muda o modo **agora** (RAM — não sobrevive ao reboot). `/etc/selinux/config` define o modo que o kernel carrega no **próximo boot**. Configurar só um dos dois perde pontos no exame.

```
# Verificar modo atual
getenforce      # Enforcing / Permissive / Disabled
sestatus        # informações detalhadas

# PASSO 1: mudar AGORA (runtime)
setenforce 1    # → Enforcing
setenforce 0    # → Permissive

# PASSO 2: tornar PERSISTENTE
vim /etc/selinux/config
# SELINUX=enforcing (ou: permissive)

# Confirmar
getenforce
```

Exemplo completo (S2-Q26 / S3-Q18): modo `enforcing`

```
setenforce 1
vim /etc/selinux/config
# SELINUX=enforcing
getenforce # Enforcing
```

Se o sistema estava em `disabled`, `setenforce 1` **não funciona** — é necessário reboot para o kernel carregar os módulos do SELinux.

Simulado	Modo pedido	Runtime	Config file
S2-Q26	enforcing	<code>setenforce 1</code>	<code>SELINUX=enforcing</code>
S3-Q18	enforcing	<code>setenforce 1</code>	<code>SELINUX=enforcing</code>
S4-Q16	permissive	<code>setenforce 0</code>	<code>SELINUX=permissive</code>
S5-Q21	enforcing	<code>setenforce 1</code>	<code>SELINUX=enforcing</code>
S6-Q25	enforcing	<code>setenforce 1</code>	<code>SELINUX=enforcing</code>

BLOCO 2 — SELinux: Contexto de Arquivos

Por que importa: o SELinux atribui uma etiqueta (contexto) a cada arquivo. Ao criar um diretório fora dos caminhos padrão, o contexto fica errado e serviços como o Apache não conseguem acessá-lo mesmo com `chmod` correto. Dois comandos são **sempre** necessários em sequência: `semanage fcontext` escreve a regra no banco de políticas; `restorecon` aplica a regra do banco ao disco.

```
# Ver contexto atual
ls -Z /arquivo
ls -Zd /diretorio

# PASSO 1: regra no banco de políticas
semanage fcontext -a -t httpd_sys_content_t "/web(/.*)?"
# -a = add | -t = tipo | regex "/web(/.*)?" = /web E tudo dentro

# PASSO 2: aplicar a regra ao disco
restorecon -Rv /web
# -R = recursivo | -v = verbose

# Verificar
ls -Z /web/content/html/index.html # deve conter: httpd_sys_content_t
```

Exemplo completo (S2-Q23): criar `/web/content/html/index.html` , contexto `httpd_sys_content_t` , persistente

```
mkdir -p /web/content/html
touch /web/content/html/index.html
semanage fcontext -a -t httpd_sys_content_t "/web(/.*)?"
restorecon -Rv /web
ls -Z /web/content/html/index.html
```

O regex `"/web(/.*)?"` cobre `/web` sozinho e tudo dentro. O `?` torna o grupo opcional.

Variação (S5-Q27): `/backup` com o mesmo contexto de `/home` (`home_root_t`)

```
mkdir /backup
ls -Zd /home # confirmar contexto de referência: home_root_t
semanage fcontext -a -t home_root_t "/backup(/.*)?"
restorecon -Rv /backup
ls -Zd /backup # deve mostrar home_root_t
```

`semanage fcontext` sem `restorecon` = regra no banco, arquivo no disco **errado**. `restorecon` sem `semanage` = muda agora, mas some no próximo relabel.

Contexto	Uso
<code>httpd_sys_content_t</code>	Conteúdo servido pelo Apache (equivalente a <code>/var/www/html</code>)
<code>home_root_t</code>	Diretórios home (<code>/home</code>)
<code>ssh_home_t</code>	Arquivos SSH (<code>~/.ssh</code>)
<code>tmp_t</code>	Diretório <code>/tmp</code>

BLOCO 3 — SELinux: Booleans

O **que são**: interruptores on/off na política SELinux — ativam comportamentos específicos sem alterar a política inteira. Ex.: mesmo com `chmod 755` correto, o SELinux bloqueia o Apache de ler arquivos NFS por padrão; o boolean `httpd_use_nfs` é a chave.

```
# Encontrar o boolean correto
semanage boolean -l | grep nfs
semanage boolean -l | grep ftp
getsebool -a | grep http

# Ativar — SEMPRE com -P no exame
setsebool -P httpd_use_nfs 1 # Apache lê arquivos NFS
setsebool -P ftpd_full_access 1 # FTP acesso total
setsebool -P httpd_can_network_connect 1 # Apache conecta à rede

# Desativar
setsebool -P httpd_use_nfs 0

# Verificar
getsebool httpd_use_nfs
# Output: httpd_use_nfs --> on
```

Exemplo completo (S5-Q26): Apache + boolean `httpd_use_nfs=on` persistente + firewall http

```
dnf install httpd -y
systemctl enable --now httpd
firewall-cmd --permanent --add-service=http && firewall-cmd --reload
setsebool -P httpd_use_nfs 1
getsebool httpd_use_nfs # httpd_use_nfs --> on
```

Variação (S6-Q29): vsftpd + boolean `ftpd_full_access=on` persistente + firewall ftp

```
dnf install vsftpd -y
systemctl enable --now vsftpd
firewall-cmd --permanent --add-service=ftp && firewall-cmd --reload
setsebool -P ftpd_full_access 1
getsebool ftpd_full_access # ftpd_full_access --> on
```

`-P` é **obrigatório** para persistência. Sem `-P` , o boolean volta ao padrão no próximo reboot — tarefa falha. Para achar o boolean certo: `semanage boolean -l | grep TERMO` — a descrição explica o que cada um faz.

BLOCO 4 — SELinux: Porta Não-Padrão (Apache porta 82)

Por que três passos: para usar porta não-padrão, três bloqueios independentes precisam ser resolvidos: 1) o serviço precisa escutar na nova porta; 2) o SELinux precisa liberar aquela porta para o tipo do serviço (`semanage port`); 3) o firewall precisa liberar a porta (`firewall-cmd`). Resolver só um ou dois não basta — o serviço fica inacessível.

Exemplo completo (S1-Q27): Apache na porta 82

```
# 1. Configuração do Apache
vim /etc/httpd/conf/httpd.conf
# Listen 80 -> Listen 82

# 2. SELinux — adicionar porta 82 ao tipo http_port_t
semanage port -a -t http_port_t -p tcp 82

# 3. Firewall — liberar porta 82
firewall-cmd --permanent --add-port=82/tcp
firewall-cmd --reload

# 4. Reiniciar Apache
```

```
systemctl restart httpd

# Verificar
semanage port -l | grep http_port_t      # deve listar porta 82
curl http://localhost:82                 # deve responder
```

Dois bloqueios independentes: SELinux (`semanage port`) e Firewall (`firewall-cmd`). Resolver só um não basta.

Referência — `semanage port` :

```
semanage port -l | grep http_port_t      # portas do Apache
semanage port -l | grep ssh_port_t       # portas do SSH
semanage port -l | grep ftp_port_t       # portas do FTP

semanage port -a -t http_port_t -p tcp 8080 # adicionar porta nova
semanage port -a -t ssh_port_t -p tcp 2222

semanage port -m -t http_port_t -p tcp 82  # modificar porta já existente com outro tipo

# forma longa (útil para copiar do man no exame)
semanage port --add --type http_port_t --proto tcp 82
```

`man semanage-port` → seção EXAMPLES tem o comando exato — use no exame para não errar a sintaxe.

BLOCO 5 — SELinux: Domínio Permissivo

Domínio permissivo vs modo global: `setenforce 0` coloca o **sistema inteiro** em permissive — todos os processos ignoram o SELinux. `semanage permissive -a httpd_t` coloca **apenas** o processo httpd em permissive — o resto do sistema continua protegido em Enforcing. Muito mais seguro para depurar um serviço específico.

Exemplo completo (S4-Q26): `httpd_t` em modo permissivo

```
semanage permissive -a httpd_t
semanage permissive -l      # deve listar httpd_t
getenforce                  # Enforcing (para todo o resto)

# Para remover:
semanage permissive -d httpd_t
```

`semanage permissive -a DOMINIO` é cirúrgico. Apenas aquele processo fica permissivo — a segurança geral do sistema é mantida.

BLOCO 6 — Firewall (firewall-cmd padrão)

A Regra dos Dois Passos do Firewall: `--permanent` escreve a regra em disco — não afeta o que está ativo agora. `--reload` carrega o arquivo com as regras permanentes para o kernel — agora está ativo. `--permanent` sem `--reload` : salvo mas não ativo. `--reload` sem `--permanent` : ativo agora, mas perde no reboot.

```
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-service={http,https}  # múltiplos de uma vez
firewall-cmd --permanent --add-service=ftp
firewall-cmd --permanent --add-service=ssh

firewall-cmd --permanent --add-port=82/tcp
firewall-cmd --permanent --add-port=2222/tcp

firewall-cmd --reload      # SEMPRE após --permanent

# Verificar
firewall-cmd --list-all
firewall-cmd --list-services
firewall-cmd --list-ports
```

Exemplo completo (S1-Q8): Apache padrão + firewall http

```
dnf install httpd -y
echo "Welcome to RHEL 10" > /var/www/html/index.html
systemctl enable --now httpd
firewall-cmd --permanent --add-service=http
firewall-cmd --reload
curl http://localhost      # verificar conteúdo
```

Simulado	Conteúdo do index.html	Serviços liberados
S1-Q8	"Welcome to RHEL 10"	<code>http</code>
S2-Q8	"Hello World!"	<code>{http,https}</code>
S3-Q13	"Welcome to the RHCSA Practice Exam!"	<code>{http,https}</code>
S4-Q13	"Welcome to the RHCSA Exam"	<code>{http,https}</code>
S5-Q11	"Hello Guys!"	<code>{http,https}</code>
S6-Q13	"Hello World!"	<code>{http,https}</code>

A partir do S2, o padrão do exame é liberar `{http,https}` de uma vez só: `firewall-cmd --permanent --add-service={http,https}` .

BLOCO 7 — Firewall: Rich Rules

Quando usar: Rich Rules permitem lógica condicional que `--add-service` não consegue — liberar um serviço apenas para uma origem específica e rejeitar para todos os outros. Útil para restringir SSH ou HTTP a uma rede corporativa.

```
# Liberar serviço apenas para rede específica
firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
    source address="192.168.1.0/24" service name="http" accept'

# Rejeitar para todos os outros
firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
    service name="http" reject'

firewall-cmd --reload
firewall-cmd --list-rich-rules
```

Exemplo completo (S5-Q28): HTTP permitido só na rede 192.168.1.0/24 , rejeitado nas demais

```
firewall-cmd --permanent --remove-service=http 2>/dev/null    # remover regra genérica, se existir

firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
    source address="192.168.1.0/24" service name="http" accept'

firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
    service name="http" reject'

firewall-cmd --reload
firewall-cmd --list-rich-rules
```

Variação (S6-Q27): mesmo padrão aplicado ao SSH

```
firewall-cmd --permanent --remove-service=ssh 2>/dev/null

firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
    source address="192.168.1.0/24" service name="ssh" accept'

firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
    service name="ssh" reject'

firewall-cmd --reload
firewall-cmd --list-rich-rules
```

Se --add-service=http (ou ssh) ainda estiver ativo, ele permite acesso de **todos**, ignorando a rich rule de reject — remova o genérico primeiro. Ordem importa: accept para rede específica deve estar antes do reject geral — o firewalld processa de cima para baixo.

🏠 Tabela Comparativa — 4 Pilares do SELinux

Situação	Ferramenta	Comando-chave	Persistente?
Mudar modo global	setenforce + config	setenforce 1 + SELINUX=enforcing	Só com config file
Contexto de arquivo	semanage fcontext + restorecon	semanage fcontext -a -t TIPO "REGEX" + restorecon -Rv /dir	Sim (banco de política)
Ligar/desligar feature	setsebool	setsebool -P BOOL 1	Só com -P
Porta não-padrão	semanage port	semanage port -a -t TIPO -p tcp PORTA	Sim (banco de política)
Isolar processo	semanage permissive	semanage permissive -a DOMÍNIO	Sim

✅ Checklist Final

SELinux Modo

- ☐ setenforce 1 (runtime) + SELINUX=enforcing no config (persistente)
- ☐ setenforce 0 (runtime) + SELINUX=permissive no config (persistente)
- ☐ getenforce e sestatus para verificar o modo atual

Contexto de Arquivos

- ☐ semanage fcontext -a -t TIPO 'REGEX' — regra no banco de políticas
- ☐ restorecon -Rv /dir — aplicar regra ao disco (SEMPRE em par)
- ☐ ls -Z para ver o contexto SELinux de arquivos

Booleans

- ☐ setsebool -P BOOLEAN 1 (-P obrigatório para persistência; output: BOOL --> on)
- ☐ getsebool BOOLEAN para verificar estado
- ☐ semanage boolean -l | grep TERMO para encontrar o boolean correto

Porta Não-Padrão

- ☐ Apache porta 82: httpd.conf (Listen 82) + semanage port + firewall-cmd
- ☐ semanage port -a -t http_port_t -p tcp 82
- ☐ man semanage-port → seção EXAMPLES para consulta no exame

Domínio Permissivo

- ☐ semanage permissive -a httpd_t — isola apenas aquele processo
- ☐ Sistema continua em Enforcing para todos os outros processos

Firewall

- ☐ firewall-cmd --permanent --add-service={http,https} && --reload
- ☐ firewall-cmd --permanent --add-port=82/tcp && --reload
- ☐ --permanent sem --reload : salvo, não ativo. --reload sem --permanent : perde no reboot

Rich Rules

- ☐ Rich rule com source address para filtrar por IP de origem

☐ Remover serviço genérico ANTES de adicionar rich rule restritiva

☐ `accept` para rede autorizada ANTES do `reject` geral